

AI WEATHERWISE API

Team Involvement in Demonstration

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Roster and suggested demo roles

Suggested roles for the final demonstration

- **S. Sathish:** opening, project objective, and overall workflow.
- **S. Sanjay:** weather API connection and weather-data retrieval.
- **C. Manikandan:** AI explanation/summary of the retrieved weather information.
- **M. Satheesh:** user interface and live input/output demonstration.
- **R. Surendar:** testing, error cases, and conclusion/future scope.

Coordination

Agree on speaking order, laptop/API setup, sample locations, request control, and a fallback demonstration if an external weather or AI service is slow. Keep API credentials private.

Contribution note

The roles above are suggested presentation/demo roles; they are not a verified record of individual coding contributions.

AI WEATHERWISE API

Scalability & Future Plan

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Reliability, scale, and feature opportunities

Current foundation

AI WeatherWise API is planned as a weather-data and AI-explanation workflow: a user provides a location, the system retrieves weather information through a weather API, and AI converts the information into a simple, user-friendly explanation.

Near-term improvements

- Add validation for location input and clear handling of invalid locations.
- Add request timeouts, retries, and useful error messages for external API failures.
- Add caching/rate limiting to reduce repeated external API requests.
- Add logging and monitoring for response time and failed requests.
- Test different locations, weather conditions, and network conditions.

Potential extensions

- Multi-day forecast and hourly forecast.
- Weather alerts and notifications.
- Multiple-language weather explanations.
- Personalized suggestions based on weather conditions.
- Historical weather summaries and simple trend visualization.

Safety and quality

Weather information should be presented with the source/time context where available, and AI-generated explanations should be treated as supportive summaries rather than a replacement for official emergency/weather alerts.

AI WEATHERWISE API

Project Demo Planning

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Suggested eight-minute agenda

Time	Activity
0:00–0:45	Title, team, problem statement and objective.
0:45–1:30	Technology stack and system workflow.
1:30–2:30	Enter a location and retrieve current weather data.
2:30–4:00	Show weather details such as temperature, condition, humidity and wind.
4:00–5:30	Show AI-generated simple explanation and weather advice.
5:30–6:30	Explain API flow and main modules.
6:30–7:15	Show validation/error handling and testing.
7:15–8:00	Future scope and questions.

Before the demo

- Confirm the application is running and the weather/AI API credentials are configured.
- Prepare 2–3 valid locations and one invalid location for demonstration.
- Hide API keys and other secrets.

Materials

Laptop, project source folder, API test/request screen if used, and screenshots of successful outputs as backup.

Success criteria

Audience understands the purpose of AI WeatherWise API, sees the weather retrieval and AI explanation flow, and understands how the external API dependencies affect the application.

AI WEATHERWISE API

Demonstration of Proposed Features

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Walkthrough supported by the project scope

Preparation

- Start the application and confirm external API configuration.
- Use safe sample locations and do not expose API credentials.

Suggested sequence

- Enter a city/location.
- Send the request to the weather API.
- Display retrieved weather information.
- Pass the relevant weather information to the AI component.
- Display a simple AI explanation and practical weather advice.
- Show validation for an invalid or unavailable location.
- Explain the main project modules and data flow.

Evidence boundary

The uploaded reference templates provide the documentation structure. Exact implementation evidence, screenshots, endpoint names, and live test results for AI WeatherWise API were not included in the supplied reference PDFs.

AI WEATHERWISE API

Communication

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Presentation flow and team coordination

Purpose

Explain how the weather API supplies data and how the AI component converts that data into a simple explanation for the user.

Suggested speakers

- **S. Sathish:** introduction, problem statement and project scope.
- **S. Sanjay:** weather API and data retrieval.
- **C. Manikandan:** AI processing and explanation.
- **M. Satheesh:** UI and live demonstration.
- **R. Surendar:** testing, future scope and conclusion.

Delivery

Show the input, API request/response or application output, weather details, and AI explanation. Avoid revealing tokens, passwords, API keys, or other secrets.

Prepare for

How is weather data retrieved? What information is sent to the AI component? What happens if the location is invalid? What happens if an external API is unavailable? How can the system be extended?

AI WEATHERWISE API

Project Executable Files

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Code layout and local setup reference

Suggested project structure

- Application entry point / main program.
- Weather API integration module.
- AI processing or explanation module.
- User interface / input-output module.
- Configuration file for API settings.
- Testing and documentation files.

Environment

Configure the weather API key and AI API key in local environment settings. Never share live secrets in screenshots, source control, or the final presentation.

Run

Use the project setup instructions and configured API credentials to start the application. Exact installation/start commands were not supplied in the reference PDFs.

API checks

Verify a valid location, weather-data retrieval, AI explanation, invalid-location handling, and external-service failure handling before the final demonstration.

AI WEATHERWISE API

Sample Project Documentation

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Project overview and workflow reference

Identity

AI WeatherWise API. Team ID SWTID-2026-3231. Team size 5. Leader S. Sathish. Members S. Sanjay, C. Manikandan, M. Satheesh, R. Surendar.

Abstract

AI WeatherWise API is a weather-information project that retrieves weather data for a user-provided location and uses AI to explain the information in simple language. The intended output can include temperature, weather condition, humidity, wind information, and useful weather advice.

Technology / workflow

Python-based application, weather API integration, AI integration, user interface, weather-data processing, and documentation/demo materials.

Core workflow

User enters location → weather API retrieves data → application processes the weather response → AI explains the weather → user sees a clear weather summary and advice.

AI WEATHERWISE API

Performance Testing

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Performance and functional evidence review

Test areas

- Location input and validation.
- Weather API response time.
- Weather-data parsing and display.
- AI response time.
- Handling of invalid locations.
- Handling of network/API failures.

Performance plan

Repeat requests in a documented environment and record sample size, latency, failure rate, network condition, location used, and the external API/AI model used.

Dependencies

Separate application processing time from weather-provider and AI-provider latency because external service response time can vary with network and provider conditions.

Limitations

No measured benchmark results or acceptance threshold for AI WeatherWise API were supplied in the uploaded reference files, so this document does not claim specific performance numbers.

AI WEATHERWISE API

No. of Functional Features Included

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Feature inventory based on project scope

Counting basis

The following capabilities represent the proposed/project workflow; this is not a code-level audit of the current implementation.

- 1. Location input
- 2. Location validation
- 3. Weather API connection
- 4. Current weather retrieval
- 5. Temperature display
- 6. Weather-condition display
- 7. Humidity display
- 8. Wind information display
- 9. Weather data processing
- 10. AI-based weather explanation
- 11. Weather advice / recommendations
- 12. Error handling for invalid or unavailable requests

Total

12 proposed user-visible/supporting capabilities are identified from the available AI WeatherWise project context.

Boundary

The exact implemented feature count should be confirmed against the current project source code before final submission.

AI WEATHERWISE API

Coding & Solution

TEAM ID	SWTID-2026-3231
TEAM SIZE	5
TEAM	S. Sathish (Leader); S. Sanjay; C. Manikandan; M. Satheesh; R. Surendar
DATE	30 September 2026

Implemented/proposed behavior supported by the project context

Weather API integration

The application accepts a user location and retrieves weather information from an external weather API.

Weather processing

The returned weather information is processed so the user can view key values such as temperature, condition, humidity, and wind information.

AI explanation

The relevant weather information is passed to an AI component to generate a simple explanation and practical weather advice.

Error handling

The project should handle invalid locations, missing responses, network failures, and unavailable external services with clear user messages.

Structure

The project can be organized into separate modules for configuration, weather API integration, AI processing, user interface, validation, and testing.

Source boundary

The uploaded reference documents are templates/examples. Exact source-code implementation details were not supplied in these PDFs.